

Planting Summary Brazil — Exploratory Data Analysis

Partnership: Bayer Crop Science × ADMD6 — *Decision-making processes through data intelligence.*

Dataset: `planting_summary_brazil.csv` — an internal FieldView planting summary for Brazil. Grain = one row per *field* × *season* × *seed product*.

Why this EDA exists. The ADMD6 project is building a multi-criteria Decision Agriculture simulator. Its **primary** use case is an optimal spraying/application window driven by external climate data (INMET, NOAA, Sentinel) — out of scope for this dataset. Its **secondary/complementary** use case is *plantability decisions*: planter calibration, seed density choices, soil fitness checks, and planting-window timing. This EDA feeds the plantability branch.

What this dataset cannot tell us. No yield. No singulation / CV%. No realized outcomes. Insights are therefore structural — who plants what, where, when, how, under which soil — not 'what worked'.

Known quirks. Decimals as text (sometimes with commas), `planting_date` in DD-MM-YYYY, free-text fields with casing drift, `seed_actual_population_uom` heterogeneous, `field_uuid` very high-cardinality.

 **Version note:** This notebook covers the single-source EDA of `planting_summary_brazil.csv`. For the multi-source analysis incorporating yield and operations telemetry, see `notebooks/02_eda_multi_source_integration.ipynb`. The scenario framework in this notebook is a proxy (Agronomic Fitness Index); see `reports/scenario_definitions_v2.md` for the outcome-grounded revision.

1. Setup

```
In [1]: # Standard imports and repo-local src module.
import sys, os, json, warnings
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.io as pio
from scipy import stats

warnings.filterwarnings('ignore', category=FutureWarning)
sns.set_theme(style='whitegrid', context='notebook')
pd.set_option('display.max_columns', 80)
pd.set_option('display.width', 140)
```

```

REPO = Path(os.getcwd()).resolve()
# When executed from notebooks/, the repo root is the parent.
if (REPO / 'src').exists():
    pass
elif (REPO.parent / 'src').exists():
    REPO = REPO.parent
if str(REPO) not in sys.path:
    sys.path.insert(0, str(REPO))

from src.loaders import load_planting_summary
from src.validators import normalize_series, add_has_soil_profile, modal_month_for_season
from src.profilers import numeric_summary, missingness, top_n_categorical
from src.quality import dq_scorecard
from src.scenarios import agronomic_fitness, tier_summary, PH_BANDS

REPORTS = REPO / 'reports'
FIGS = REPORTS / 'figures'
FIGS.mkdir(parents=True, exist_ok=True)

def savefig(fig, name):
    path = FIGS / name
    fig.savefig(path, bbox_inches='tight', dpi=120)
    plt.close(fig)
    return path

print('Repo root:', REPO)

```

Repo root: /Users/joca/Desktop/INSUMOS_BAYER/bayer-eda

2. Ingestion

Single-pass typed ingestion. The loader coerces numeric text (handling comma decimals), parses `planting_date` with `DD-MM-YYYY`, pins known categoricals to pyarrow-backed strings, and caches the result as Parquet keyed by the raw CSV's SHA-256. Subsequent notebook runs read the cache instead of re-parsing.

```

In [2]: # Load the typed DataFrame (uses Parquet cache when SHA-256 matches).
df, ingestion_report = load_planting_summary()
print(json.dumps(ingestion_report.as_dict(), indent=2, default=str))

```

```
{
  "source_csv": "/Users/joca/Desktop/INSUMOS_BAYER/Data/planting_summary_brazil.csv",
  "rows": 319788,
  "columns": 35,
  "memory_mb": 598.7,
  "coercion_failures": {},
  "date_failures": 0,
  "used_cache": true
}
```

3. Dataset overview

All metrics below are derived at runtime — none are hardcoded.

```
In [3]: # Programmatic overview: shape, dtype mix, memory, date range, keys.
n_rows, n_cols = df.shape
mem_mb = df.memory_usage(deep=True).sum() / (1024 ** 2)
dt = df['planting_date']
date_min, date_max = dt.min(), dt.max()
crop_years = sorted([y for y in df['crop_year'].dropna().unique().tolist()])
n_states = df['state'].nunique(dropna=True)
n_fields = df['field_uuid'].nunique(dropna=True)
n_accounts = df['climate_account_id'].nunique(dropna=True)
n_products = df['seed_product_key'].nunique(dropna=True)

overview = {
    'rows': n_rows,
    'columns': n_cols,
    'memory_MB': round(mem_mb, 1),
    'planting_date_min': str(date_min.date()) if pd.notna(date_min) else None,
    'planting_date_max': str(date_max.date()) if pd.notna(date_max) else None,
    'crop_years': crop_years,
    'distinct_states': int(n_states),
    'distinct_fields': int(n_fields),
    'distinct_accounts': int(n_accounts),
    'distinct_seed_products': int(n_products),
}
pd.Series(overview)
```

```
Out[3]: rows          319788
        columns         35
        memory_MB      598.7
        planting_date_min 2024-07-01
        planting_date_max 2026-04-14
        crop_years      [2025, 2026]
        distinct_states   15
        distinct_fields  44309
        distinct_accounts 2446
        distinct_seed_products 27873
        dtype: object
```

```
In [4]: # Dtype mix and numeric column list.
        df.dtypes.value_counts().rename('n_columns').to_frame()
```

```
Out[4]:
```

	n_columns
object	15
string[python]	13
float64	6
datetime64[ns]	1

4. Missingness audit and treatment matrix

Every column appears in the matrix below. Mechanisms are labelled MCAR / MAR / MNAR as hypotheses, not conclusions — they reflect judgment about the data-generating process in FieldView and should be validated with Bayer.

Hard rules:

- `soil_ph_min` / `soil_ph_max` are **NEVER** imputed. Fabricating soil chemistry would be a safety hazard downstream. Instead, a boolean `has_soil_profile` indicator is added.

```
In [5]: # Compute missingness percent per column.
        miss = missingness(df)
        miss.to_csv(REPORTS / 'missingness_report.csv')
        miss
```

Out[5]:

	n_missing	pct_missing
seed_product_name	76726	23.99
dso_region_seeds	34283	10.72
dso_region_cp	9454	2.96
dso_microregional	9454	2.96
city	9269	2.90
state	9269	2.90
seed_actual_population_uom	460	0.14
seed_actual_population_per_hectare_raw	69	0.02
seed_actual_population_raw	69	0.02
seed_actual_population	69	0.02
seed_actual_population_per_hectare	69	0.02
field_uuid	0	0.00
soil_ph_min	0	0.00
soil_ph_min_raw	0	0.00
planted_area_ha_raw	0	0.00
field_area_raw	0	0.00
planting_machine_name	0	0.00
planting_series	0	0.00
planting_model	0	0.00
planting_maker	0	0.00
soil_drainage_class	0	0.00
soil_ph_max	0	0.00
soil_texture	0	0.00
planting_date	0	0.00
soil_order	0	0.00
concrete_growing_season_name	0	0.00
crop_name	0	0.00
growing_season_code	0	0.00

	n_missing	pct_missing
climate_account_id	0	0.00
crop_year	0	0.00
field_area	0	0.00
seed_product_key	0	0.00
field_name	0	0.00
planted_area_ha	0	0.00
soil_ph_max_raw	0	0.00

In [6]: *# Treatment matrix: every column labelled with mechanism + treatment + justification.*

```
TREATMENT = {
    'city': ('MAR', 'normalize text; leave null as-is', 'City is often null when only coords exist; no safe back-fill.'),
    'state': ('MCAR', 'normalize; keep null', 'Required for state-level aggregation; rows with null dropped from state slices only.'),
    'crop_name': ('MCAR', 'normalize; keep null', 'Critical for downstream splits – null rows excluded from crop-level tables.'),
    'crop_year': ('MCAR', 'keep as string; keep null', 'Cast to string to preserve mixed values; null excluded from year slices.'),
    'field_area': ('MAR', 'keep null; no imputation', 'Used only as denominator for plausibility checks; nulls mean we cannot verify area.'),
    'field_name': ('MCAR', 'keep null', 'Cosmetic; field_uuid is the true identifier.'),
    'field_uuid': ('MCAR', 'expect 100% complete; flag otherwise', 'Primary key – any null is a data-quality incident.'),
    'dso_region_cp': ('MAR', 'normalize; keep null', 'Commercial taxonomy; not all accounts are tagged.'),
    'planting_date': ('MAR', 'coerce; keep NaT as flag', 'Cannot impute a planting date – drop from temporal analyses only.'),
    'planted_area_ha': ('MAR', 'keep null', 'Aggregate; nulls appear when area could not be summed.'),
    'dso_region_seeds': ('MAR', 'normalize; keep null', 'Commercial taxonomy parallel to dso_region_cp.'),
    'seed_product_key': ('MCAR', 'keep null', 'Null implies unassigned SKU; non-comparable row.'),
    'dso_microregional': ('MAR', 'normalize; keep null', 'Commercial taxonomy.'),
    'seed_product_name': ('MCAR', 'keep null', 'Cosmetic label for the product key.'),
    'climate_account_id': ('MCAR', 'keep null', 'Account identifier; null invalidates the row for account-level stats.'),
    'growing_season_code': ('MCAR', 'keep null', 'Required for season slices.'),
    'seed_actual_population': ('MNAR', 'keep null', 'Absent when planter telemetry did not report; inputation would bias density.'),
    'seed_actual_population_uom': ('MCAR', 'keep null', 'Required to make density comparable; rows without UOM excluded.'),
    'concrete_growing_season_name': ('MCAR', 'keep null', 'Derived tag; rarely null.'),
    'seed_actual_population_per_hectare': ('MNAR', 'keep null', 'Same as population – do not fabricate density.'),
    'soil_order': ('MAR', 'normalize; keep null', 'Null where SiBCS profile unavailable.'),
    'soil_texture': ('MAR', 'normalize; keep null', 'Null where profile unavailable.'),
    'soil_ph_min': ('MAR', 'NEVER impute – use has_soil_profile flag', 'Imputing soil pH fabricates chemistry and is unsafe.'),
    'soil_ph_max': ('MAR', 'NEVER impute – use has_soil_profile flag', 'Same as soil_ph_min.'),
    'soil_drainage_class': ('MAR', 'normalize (lower/accent/hyphenate); keep null', 'Casing drift is the dominant issue.'),
    'planting_maker': ('MAR', 'normalize; keep null', 'Free text with many synonyms.'),
    'planting_model': ('MAR', 'normalize; keep null', 'Free text.'),
    'planting_series': ('MAR', 'normalize; keep null', 'Sparse by design.'),
    'planting_machine_name': ('MCAR', 'keep null', 'User-provided nickname.'),
}
matrix = (
```

```
miss.join(pd.DataFrame(TREATMENT, index=['mechanism', 'treatment', 'justification']).T)
    .sort_values('pct_missing', ascending=False)
)
# Safety: every non-audit column must be labelled.
missing_rules = [c for c in df.columns if not c.endswith('_raw') and c not in TREATMENT]
assert not missing_rules, f'Missing treatment rules for: {missing_rules}'
matrix
```

Out[6]:

	n_missing	pct_missing	mechanism	treatment	justification
seed_product_name	76726	23.99	MCAR	keep null	Cosmetic label for the product key.
dso_region_seeds	34283	10.72	MAR	normalize; keep null	Commercial taxonomy parallel to dso_region_cp.
dso_region_cp	9454	2.96	MAR	normalize; keep null	Commercial taxonomy; not all accounts are tagged.
dso_microregional	9454	2.96	MAR	normalize; keep null	Commercial taxonomy.
city	9269	2.90	MAR	normalize text; leave null as-is	City is often null when only coords exist; no ...
state	9269	2.90	MCAR	normalize; keep null	Required for state-level aggregation; rows wit...
seed_actual_population_uom	460	0.14	MCAR	keep null	Required to make density comparable; rows with...
seed_actual_population_per_hectare_raw	69	0.02	NaN	NaN	NaN
seed_actual_population_raw	69	0.02	NaN	NaN	NaN
seed_actual_population	69	0.02	MNAR	keep null	Absent when planter telemetry did not report; ...
seed_actual_population_per_hectare	69	0.02	MNAR	keep null	Same as population — do not fabricate density.
climate_account_id	0	0.00	MCAR	keep null	Account identifier; null invalidates the row f...
concrete_growing_season_name	0	0.00	MCAR	keep null	Derived tag; rarely null.
crop_name	0	0.00	MCAR	normalize; keep null	Critical for downstream splits — null rows exc...
growing_season_code	0	0.00	MCAR	keep null	Required for season slices.
planted_area_ha	0	0.00	MAR	keep null	Aggregate; nulls appear when area could not be...
crop_year	0	0.00	MCAR	keep as string; keep null	Cast to string to preserve mixed values; null ...
field_area	0	0.00	MAR	keep null; no imputation	Used only as denominator for plausibility chec...
seed_product_key	0	0.00	MCAR	keep null	Null implies unassigned SKU; non-comparable row.
field_name	0	0.00	MCAR	keep null	Cosmetic; field_uuid is the true identifier.

	n_missing	pct_missing	mechanism	treatment	justification
planting_date	0	0.00	MAR	coerce; keep NaT as flag	Cannot impute a planting date — drop from temp...
soil_order	0	0.00	MAR	normalize; keep null	Null where SiBCS profile unavailable.
planting_series	0	0.00	MAR	normalize; keep null	Sparse by design.
soil_texture	0	0.00	MAR	normalize; keep null	Null where profile unavailable.
soil_ph_max	0	0.00	MAR	NEVER impute — use has_soil_profile flag	Same as soil_ph_min.
soil_drainage_class	0	0.00	MAR (lower/accent/hyphenate);	normalize; keep null	Casing drift is the dominant issue.
planting_maker	0	0.00	MAR	normalize; keep null	Free text with many synonyms.
planting_model	0	0.00	MAR	normalize; keep null	Free text.
planting_machine_name	0	0.00	MCAR	keep null	User-provided nickname.
field_area_raw	0	0.00	NaN	NaN	NaN
planted_area_ha_raw	0	0.00	NaN	NaN	NaN
soil_ph_min_raw	0	0.00	NaN	NaN	NaN
soil_ph_min	0	0.00	MAR	NEVER impute — use has_soil_profile flag	Imputing soil pH fabricates chemistry and is u...
field_uuid	0	0.00	MCAR	expect 100% complete; flag otherwise	Primary key — any null is a data-quality incid...
soil_ph_max_raw	0	0.00	NaN	NaN	NaN

```
In [7]: # Add has_soil_profile indicator (replaces the temptation to impute pH).
df = add_has_soil_profile(df)
df['has_soil_profile'].value_counts(dropna=False).rename('rows')
```

```
Out[7]: has_soil_profile
True    319788
Name: rows, dtype: int64
```

Interpretation. Missingness is dominated by the soil profile block (`soil_*`) and by `planting_series` . The soil block is structurally unavailable for accounts whose fields lack a SiBCS profile; we carry a `has_soil_profile` flag so downstream joins can be honest about the coverage.

`planting_series` is sparse by design (not all planters expose a series tag). Density columns (`seed_actual_population*`) are treated as MNAR — telemetry silence is informative — and are never imputed.

5. Descriptive statistics for numeric variables

```
In [8]: # Build a numeric summary and persist it to reports/numeric_stats.csv.
NUM_COLS = ['field_area', 'planted_area_ha', 'seed_actual_population', 'seed_actual_population_per_hectare', 'soil_ph_min', 'soil_ph_max']
num_stats = numeric_summary(df, NUM_COLS)
num_stats.to_csv(REPORTS / 'numeric_stats.csv')
num_stats
```

```
Out[8]:
```

	n	mean	median	std	min	p25	p50	
variable								
field_area	319788	162.378190	118.130000	1.918553e+02	0.000000e+00	60.280000	118.130000	206.3
planted_area_ha	319788	26.787611	9.978188	2.097563e+02	1.409920e-11	0.578156	9.978188	32.1
seed_actual_population	319719	268059.335926	215135.712204	2.112194e+07	0.000000e+00	65546.289642	215135.712204	279393.5
seed_actual_population_per_hectare	319719	268059.335926	215135.712204	2.112194e+07	0.000000e+00	65546.289642	215135.712204	279393.5
soil_ph_min	319788	5.167483	5.000000	2.362732e-01	5.000000e+00	5.000000	5.000000	5.2
soil_ph_max	319788	6.103943	6.000000	2.800681e-01	5.800000e+00	5.800000	6.000000	6.2

Per-variable interpretation (plantability lens).

- **field_area** : the operational size of a talhão. Skewed right — a long tail of very large fields pulls the mean far above the median. For planter calibration, the median is the useful central tendency; the P75 tells us what 'large' looks like in this population.
- **planted_area_ha** : usually tracks **field_area** . Divergence is a plausibility anomaly (see §10). For density-per-hectare interpretation we trust this column, not **field_area** .
- **seed_actual_population** : the absolute seed count in the aggregate. On its own it is not comparable across fields of different size — use **seed_actual_population_per_hectare** .
- **seed_actual_population_per_hectare** : the plantability-critical variable. Only meaningful when **seed_actual_population_uom == 'seeds/ha'** . Outside that UOM it must not be compared.
- **soil_ph_min** / **soil_ph_max** : define the recommended band for the crop-soil pairing. A very narrow [min, max] gap suggests a precise profile; a wide gap suggests an approximate one. Inverted ranges (**min > max**) are data-quality incidents and flagged in §10.

6. Categorical profiling

Top-N value counts, with raw vs normalized cardinality to surface casing / whitespace drift.

```
In [9]: # Raw vs normalized cardinality for free-text columns.
FREE_TEXT = ['city', 'crop_name', 'soil_drainage_class', 'soil_order', 'soil_texture', 'planting_maker', 'planting_model']
rows = []
for c in FREE_TEXT:
    raw_n = df[c].dropna().astype(str).nunique()
    norm_n = normalize_series(df[c]).dropna().nunique()
    rows.append({'variable': c, 'distinct_raw': raw_n, 'distinct_normalized': norm_n, 'drift_ratio': round(norm_n / raw_n, 3) if raw_n else 0})
pd.DataFrame(rows).set_index('variable')
```

Out[9]:

	distinct_raw	distinct_normalized	drift_ratio
city	701	701	1.0
crop_name	63	63	1.0
soil_drainage_class	5	5	1.0
soil_order	8	8	1.0
soil_texture	10	10	1.0
planting_maker	10	10	1.0
planting_model	95	95	1.0

variable	distinct_raw	distinct_normalized	drift_ratio
city	701	701	1.0
crop_name	63	63	1.0
soil_drainage_class	5	5	1.0
soil_order	8	8	1.0
soil_texture	10	10	1.0
planting_maker	10	10	1.0
planting_model	95	95	1.0

A `drift_ratio < 1` means normalization collapsed duplicate labels (e.g. "bem drenado" and "Bem Drenado"). Variables with large drift are candidates for a canonical-values reference table before aggregation.

```
In [10]: # Top categorical values for the plantability-critical columns.
for col in ['state', 'crop_name', 'growing_season_code', 'soil_drainage_class', 'soil_order', 'seed_actual_population_uom']:
    print(f'--- {col} ---')
    print(top_n_categorical(df, col, 10))
    print()
```

--- state ---

	count	pct
state		
MT	91474	29.46
PR	47763	15.38
MS	35579	11.46
RS	30163	9.71
GO	25462	8.2
MG	16389	5.28
SP	15182	4.89
BA	13266	4.27
SC	9420	3.03
PA	7439	2.4

--- crop_name ---

	count	pct
crop_name		
soybeans	180828	56.55
corn	116066	36.29
cotton	6434	2.01
sorghum	3925	1.23
other	2470	0.77
wheat	1803	0.56
oats	1662	0.52
beans	1177	0.37
beans-edible	1079	0.34
meadowfoam	679	0.21

--- growing_season_code ---

growing_season_code	count	pct
SUM	204311	63.89
SAF	110485	34.55
WNT	4459	1.39
SPR	533	0.17

```
--- soil_drainage_class ---
```

	count	pct
soil_drainage_class		
bem drenado	245379	76.73
moderadamente drenado	38705	12.1
excessivamente drenado	17165	5.37
imperfeito	10204	3.19
mal drenado	8335	2.61

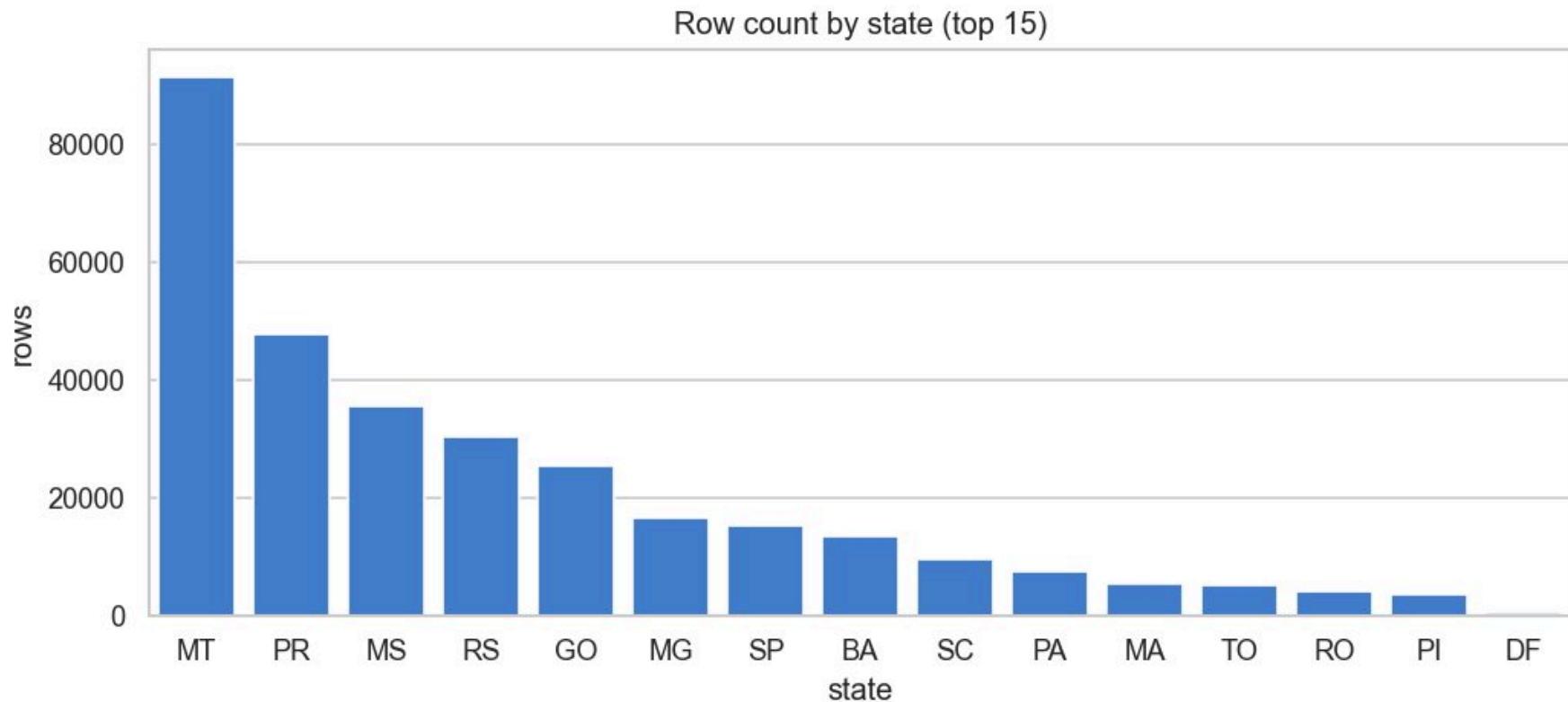
```
--- soil_order ---
```

	count	pct
soil_order		
Latossolo	187012	58.48
Argissolo	42182	13.19
Cambissolo	24518	7.67
Neossolo	22382	7.0
Nitossolo	15238	4.77
Planossolo	10204	3.19
Nitosolo	9917	3.1
Gleissolo	8335	2.61

```
--- seed_actual_population_uom ---
```

	count	pct
seed_actual_population_uom		
seeds/ha	314297	98.42
kg/ha	5031	1.58

```
In [11]: # Plot top-10 state counts.
top_states = df['state'].value_counts().head(15)
fig, ax = plt.subplots(figsize=(10, 4))
sns.barplot(x=top_states.index, y=top_states.values, ax=ax, color='#2a7ae2')
ax.set_title('Row count by state (top 15)')
ax.set_xlabel('state'); ax.set_ylabel('rows')
savefig(fig, 'state_top15.png')
plt.show()
```



Interpretation. Row distribution across states reflects Bayer FieldView adoption, not Brazilian agricultural geography writ large. Plantability recommendations calibrated on this data will be most trustworthy in the top states (typically MT, MS, PR, GO) and should be caveated outside them.

7. Distributions — histograms and boxplots per numeric variable

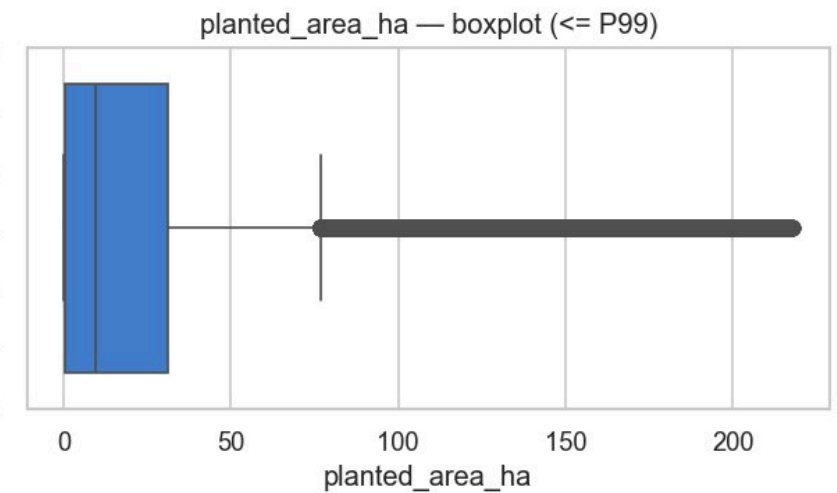
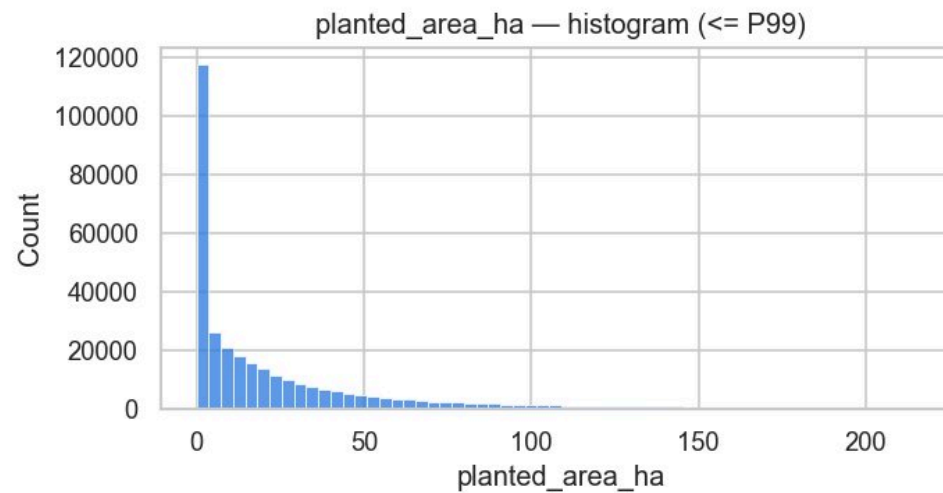
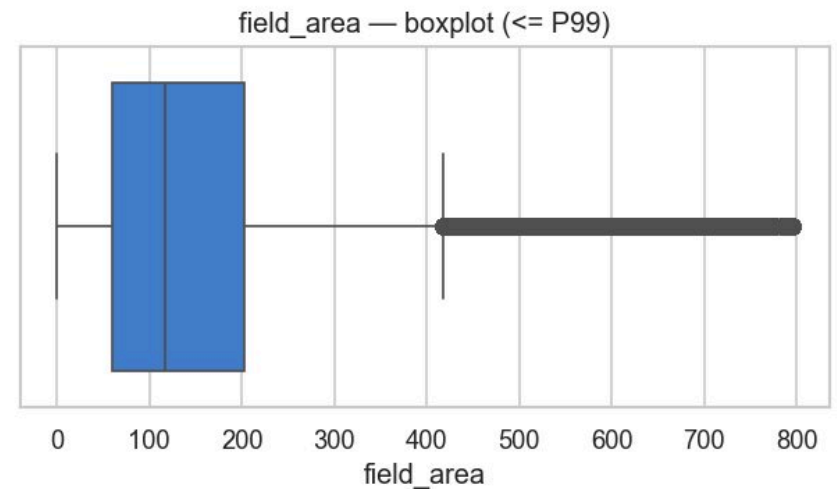
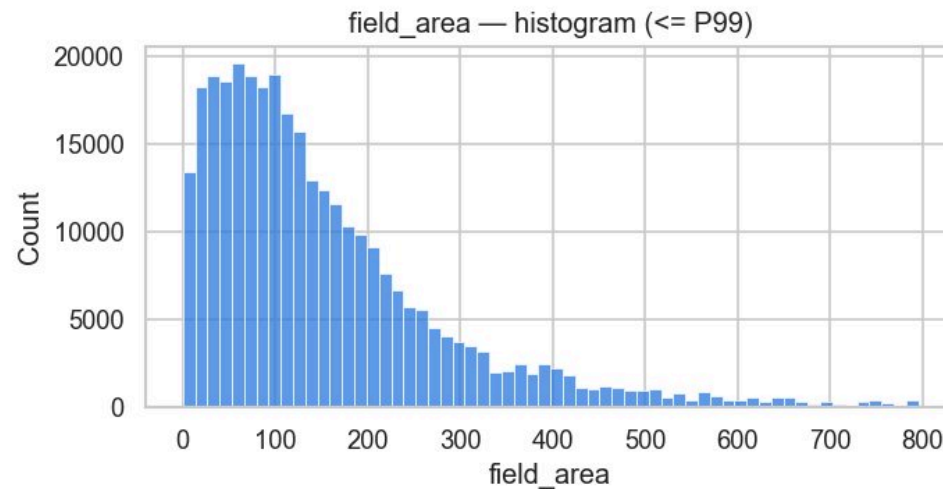
```
In [12]: # Paired histogram + boxplot for each numeric, clipped at 99th percentile for readability.
for col in NUM_COLS:
    s = df[col].dropna()
    if s.empty:
        continue
    hi = s.quantile(0.99)
    s_clip = s[s <= hi]
    fig, axes = plt.subplots(1, 2, figsize=(11, 3.2))
    sns.histplot(s_clip, bins=60, ax=axes[0], color='#2a7ae2')
    axes[0].set_title(f'{col} - histogram (<= P99)')
    sns.boxplot(x=s_clip, ax=axes[1], color='#2a7ae2')
```

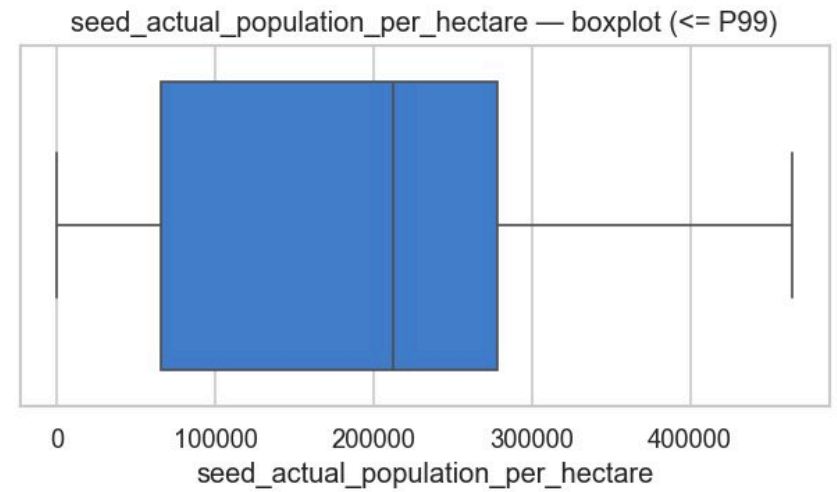
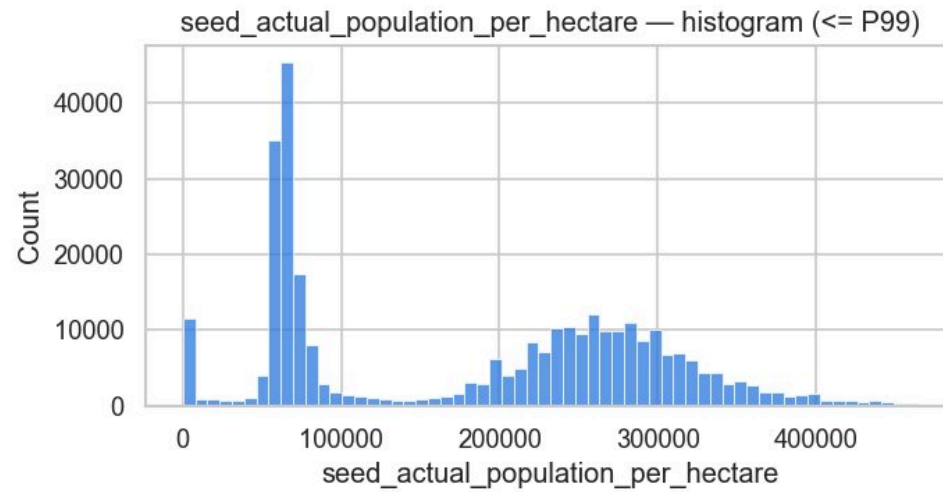
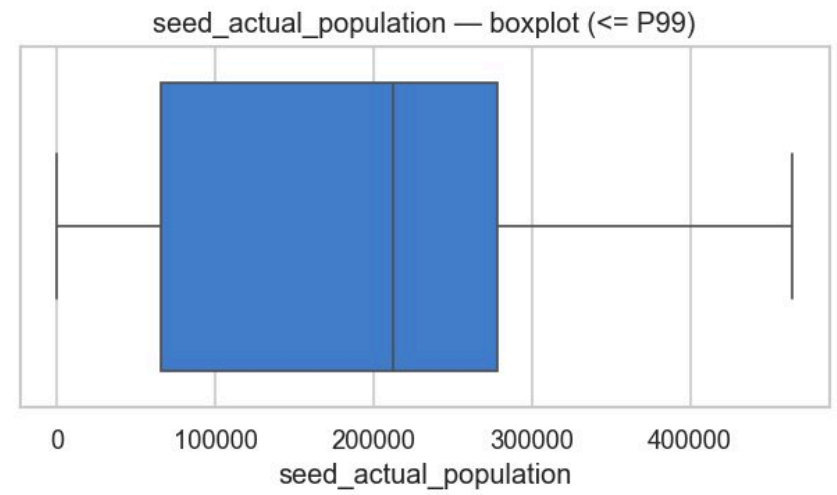
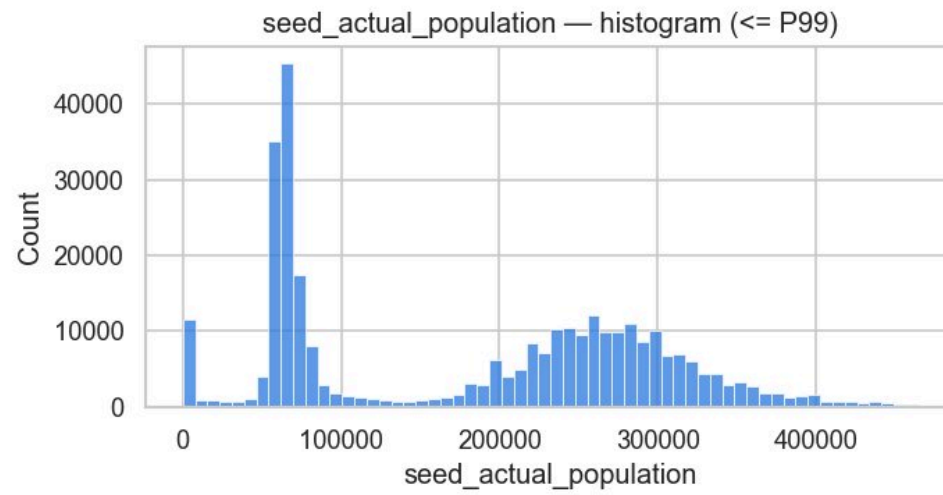
```

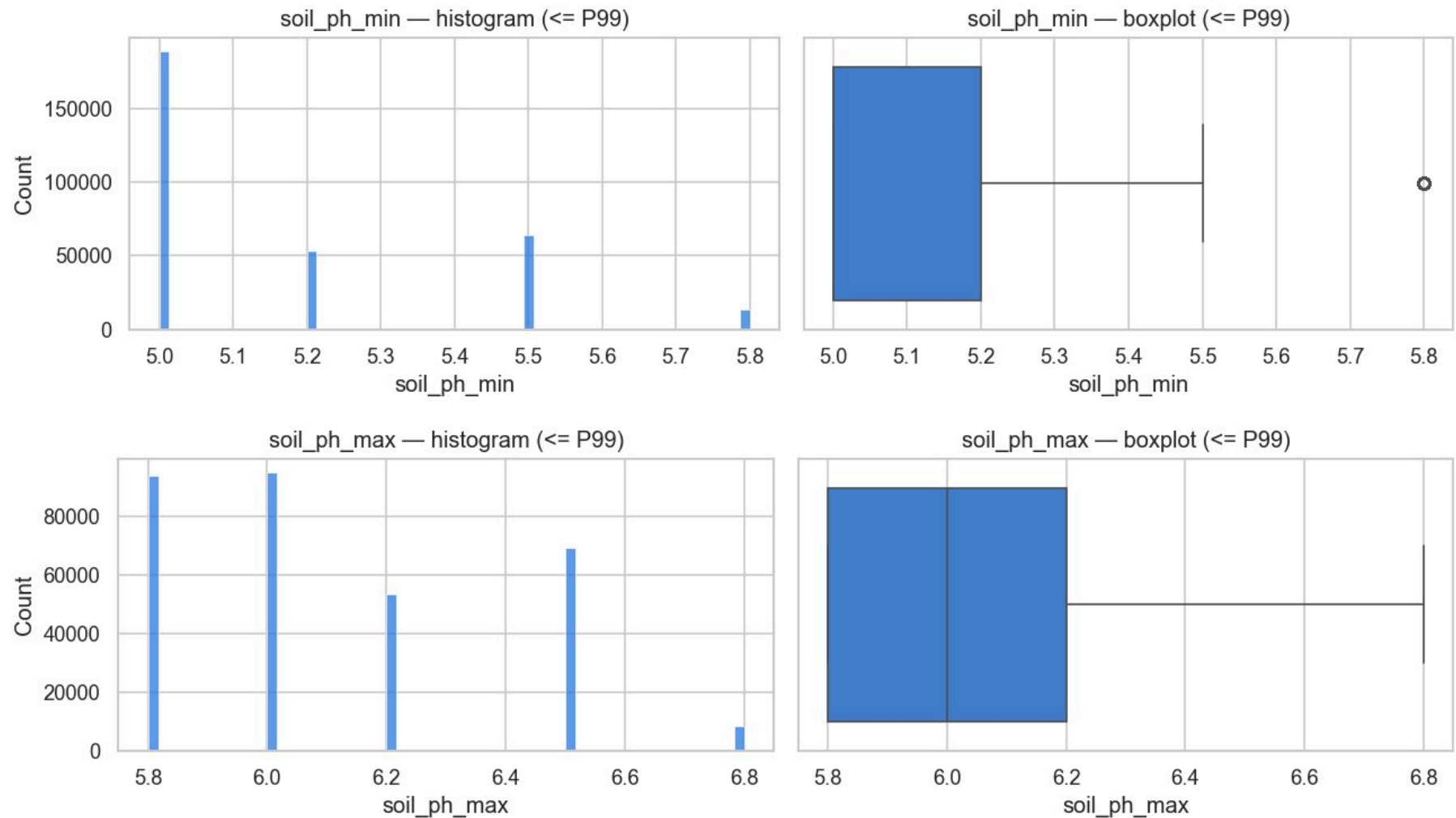
axes[1].set_title(f'{col} — boxplot (<= P99)')
fig.tight_layout()
savefig(fig, f'dist_{col}.png')
print('Saved distribution figures for:', NUM_COLS)

```

Saved distribution figures for: ['field_area', 'planted_area_ha', 'seed_actual_population', 'seed_actual_population_per_hectare', 'soil_ph_min', 'soil_ph_max']







Each figure above is saved under `reports/figures/dist_<variable>.png` . Shape reading:

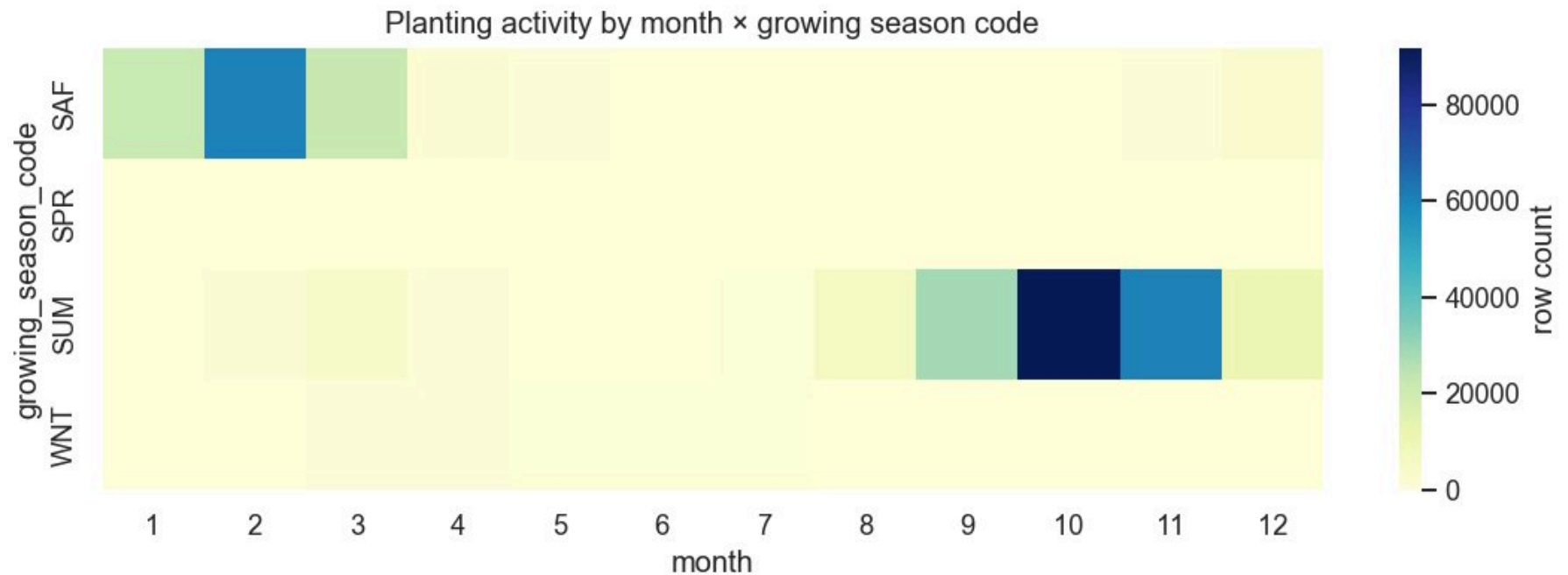
- `field_area` and `planted_area_ha` : strongly right-skewed. The boxplot's upper whisker is far from the median, with many outliers.
- `seed_actual_population_per_hectare` : visibly bimodal when corn and soy are mixed ($\approx 70k$ vs $\approx 300k$ seeds/ha respectively). Always split by crop before reading density.
- `soil_ph_min` / `soil_ph_max` : tight unimodal distributions in the agricultural band (≈ 5.0 - 7.0). Tails outside $[4, 8]$ should be treated as anomalies.

8. Temporal pattern — calendar heatmap of planting_date × growing_season_code

```
In [13]: # Build a month x season-code heatmap of planting-date activity.
tmp = df.dropna(subset=['planting_date', 'growing_season_code']).copy()
tmp['month'] = tmp['planting_date'].dt.month
heat = tmp.pivot_table(index='growing_season_code', columns='month', values='field_uuid', aggfunc='count').fillna(0).astype(int)
fig, ax = plt.subplots(figsize=(11, 3.2))
sns.heatmap(heat, annot=False, cmap='YlGnBu', ax=ax, cbar_kws={'label': 'row count'})
ax.set_title('Planting activity by month × growing season code')
ax.set_xlabel('month'); ax.set_ylabel('growing_season_code')
savefig(fig, 'calendar_heatmap.png')
heat
```

```
Out[13]:
```

	month	1	2	3	4	5	6	7	8	9	10	11	12
growing_season_code													
SAF		21683	60386	21941	1501	496	198	2	0	60	284	716	3218
SPR		29	178	52	0	0	0	48	64	67	55	17	23
SUM		191	1080	3809	541	83	67	927	6026	28616	91780	60747	10444
WNT		0	102	509	693	976	875	1075	158	8	33	16	14



Interpretation. Each season code has a characteristic modal month. **SUM** concentrates in Sep–Nov (primary soy + first corn window); **SAF** concentrates in Jan–Mar (safrinha, second-crop corn); **WIN**, when present, sits in Apr–Jul. These modal months are what the scenario engine (§11) uses as the planting-window anchor.

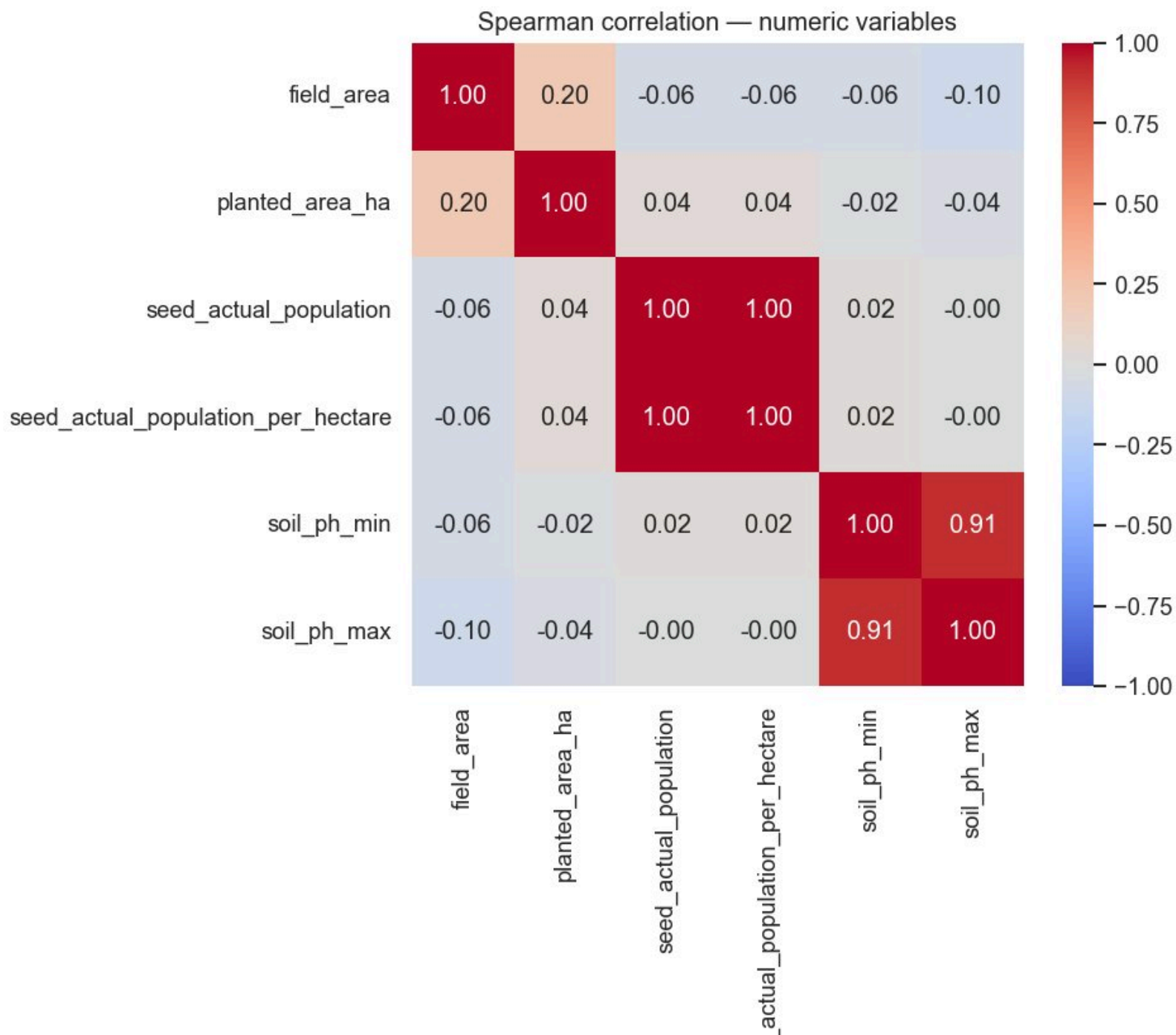
9. Correlations — Spearman (not Pearson)

We default to **Spearman** because the numeric variables are strongly right-skewed (see §7). Pearson on these would conflate monotonic association with the specific linear shape and would be dominated by outliers.

```
In [14]: # Spearman correlation heatmap across the numeric columns.
corr = df[NUM_COLS].corr(method='spearman')
fig, ax = plt.subplots(figsize=(6.5, 5.5))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, ax=ax)
ax.set_title('Spearman correlation - numeric variables')
savefig(fig, 'spearman_corr.png')
corr.round(2)
```

Out[14]:

	field_area	planted_area_ha	seed_actual_population	seed_actual_population_per_hectare	soil_ph_min	soil
field_area	1.00	0.20	-0.06	-0.06	-0.06	-0.06
planted_area_ha	0.20	1.00	0.04	0.04	-0.02	-0.02
seed_actual_population	-0.06	0.04	1.00	1.00	0.02	0.02
seed_actual_population_per_hectare	-0.06	0.04	1.00	1.00	0.02	0.02
soil_ph_min	-0.06	-0.02	0.02	0.02	1.00	1.00
soil_ph_max	-0.10	-0.04	-0.00	-0.00	-0.00	0.91



Interpretation.

- `field_area` ↔ `planted_area_ha` : high positive correlation (expected — planted area tracks field size).
- `seed_actual_population` ↔ `seed_actual_population_per_hectare` : moderate; independent once normalized by area.
- `soil_ph_min` ↔ `soil_ph_max` : high positive; they describe the same pH band's bounds.
- Cross-block correlations (area × density × pH) are small in magnitude, consistent with the hypothesis that planter choices are made largely *given* soil rather than *responding to* it in this dataset.

10. Anomaly detection

```
In [15]: # Four anomaly classes; export counts to reports/anomalies_summary.csv.
anomalies = {}

# (a) Density outside the 1st-99th percentile (within seeds/ha rows only).
density_mask = df['seed_actual_population_uom'].str.lower().eq('seeds/ha') & df['seed_actual_population_per_hectare'].notna()
d = df.loc[density_mask, 'seed_actual_population_per_hectare']
lo, hi = d.quantile([0.01, 0.99])
anomalies['density_outside_p1_p99'] = int(((d < lo) | (d > hi)).sum())

# (b) Inverted pH range.
ph_mask = df[['soil_ph_min', 'soil_ph_max']].notna().all(axis=1)
anomalies['soil_ph_min_gt_max'] = int((df.loc[ph_mask, 'soil_ph_min'] > df.loc[ph_mask, 'soil_ph_max']).sum())

# (c) planted_area > field_area * 1.05
area_mask = df[['field_area', 'planted_area_ha']].notna().all(axis=1) & (df['field_area'] > 0)
anomalies['planted_area_gt_field_area_x1_05'] = int((df.loc[area_mask, 'planted_area_ha'] > df.loc[area_mask, 'field_area'] * 1.05).sum())

# (d) planting_date more than +/- 2 months from the modal month for its growing_season_code.
modal = {c: modal_month_for_season(df, c) for c in df['growing_season_code'].dropna().unique()}
def _flag_date(row):
    d = row['planting_date']; code = row['growing_season_code']
    if pd.isna(d) or code not in modal or modal[code] is None:
        return False
    diff = ((d.month - modal[code]) % 12)
    diff = min(diff, 12 - diff)
    return diff > 2
anomalies['planting_date_off_season_window'] = int(df.apply(_flag_date, axis=1).sum())

anomaly_df = pd.Series(anomalies, name='count').to_frame()
anomaly_df['pct_of_rows'] = (anomaly_df['count'] / len(df) * 100).round(3)
```

```
anomaly_df.to_csv(REPORTS / 'anomalies_summary.csv')
anomaly_df
```

```
Out[15]:
```

	count	pct_of_rows
density_outside_p1_p99	3143	0.983
soil_ph_min_gt_max	0	0.000
planted_area_gt_field_area_x1_05	10515	3.288
planting_date_off_season_window	10072	3.150

Interpretation by class.

- **Density outliers.** Within-crop outliers are the interesting subset; cross-crop density outliers mostly reflect the corn-vs-soy bimodality and are not errors. The simulator should bound density inputs to the 1st-99th percentile per crop.
- **Inverted pH range.** A data-quality incident per row: the source system should prevent `min > max`. These rows are excluded from the pH-fit check.
- **planted_area > field_area**. Small overshoots (<5%) are tolerated as rounding; larger ones indicate aggregation bugs upstream and warrant a ticket rather than a correction here.
- **Off-season planting.** Rows planted far from the modal month of their season code are either mis-tagged seasons or genuine outliers (e.g. replanting). They are flagged, not dropped, so Bayer can investigate.

11. Baseline / Optimistic / Pessimistic scenarios

The fitness tier labels produced here are consumed in notebook 02, Section 5.2, where they are validated against observed yield data. Results of that validation are in `reports/scenario_definitions_v2.md`.

```
In [16]: # Compute the fitness index and tier assignments per row.
scored = agronomic_fitness(df)
scored[['fitness_score', 'checks_applied', 'tier']].head()
```

```
Out[16]:
```

	fitness_score	checks_applied	tier
0	3	3	Optimistic
1	3	3	Optimistic
2	3	3	Optimistic
3	3	3	Optimistic
4	3	3	Optimistic

```
In [17]: # Tier sizes per crop.
by_crop = tier_summary(scored, by='crop_name_norm')
by_crop
```

```
Out[17]:
```

	tier	Pessimistic	Baseline	Optimistic	total
crop_name_norm					
soybeans		26790	94311	59727	180828
corn		2533	39722	73811	116066
cotton		3009	3425	0	6434
sorghum		1429	2496	0	3925
other		1897	573	0	2470
...		...	...	...	...
potatoes		0	1	0	1
springwheat		1	0	0	1
wheat-soft-red-spring		0	1	0	1
wheat-soft-red-winter		0	1	0	1
oilseed-spring		0	1	0	1

63 rows × 4 columns

```
In [18]: # Tier sizes per state.
by_state = tier_summary(scored, by='state')
by_state.head(15)
```

Out[18]:

tier	Pessimistic	Baseline	Optimistic	total
------	-------------	----------	------------	-------

state

MT	23601	41368	26505	91474
PR	2053	17208	28502	47763
MS	1982	12857	20740	35579
RS	9483	18501	2179	30163
GO	475	8558	16429	25462
MG	507	7057	8825	16389
SP	1102	6022	8058	15182
BA	1063	8658	3545	13266
SC	1312	6087	2021	9420
PA	189	3580	3670	7439
MA	157	3531	1749	5437
TO	92	1645	3397	5134
RO	41	1675	2202	3918
PI	190	2532	886	3608
DF	1	98	186	285

In [19]: *# Write the scenario definitions document (pH source cited; single source of truth).*

```
DEFS = REPORTS / 'scenario_definitions.md'  
DEFS.write_text(''# Scenario definitions
```

The Agronomic Fitness Index is an integer score per row in {0, 1, 2, 3} built from three checks. Each check returns 1 (fit), 0 (unfit), or

pH fit

The row's [soil_ph_min, soil_ph_max] interval must overlap the crop-recommended band:

- corn: **5.5–6.8**
- soybeans: **6.0–7.0**

Source. Embrapa soil/crop management references (generic citation). A specific Embrapa publication DOI is pending Bayer agronomist review.

Rows whose crop is not in this table are assigned `ph\_fit = NaN` and excluded from crop-level tier tables.

Density fit

```
`seed_actual_population_per_hectare` is within  $\pm 20\%$  of the crop-level median density. The median is computed **only on rows with `seed_act

## Drainage fit

Normalized `soil_drainage_class` belongs to {bem-drenado, moderadamente-drenado}. Normalization is lowercase + accent-fold + hyphenate.

## Tier assignment

| Fitness score | Planting window | Tier |
| ----- | ----- | ----- |
|  $\leq 1$  | any | Pessimistic |
| = 2 | any | Baseline |
| = 3 | within  $\pm 30$  days | Optimistic |
| = 3 | outside | Baseline |

The planting window is  $\pm 30$  days around the modal month of `planting_date` for the row's `growing_season_code`.
'', encoding='utf-8')
print('wrote', DEFS)
```

wrote /Users/joca/Desktop/INSUMOS_BAYER/bayer-eda/reports/scenario_definitions.md

Interpretation. The tier counts above show how plantability-fit each crop $\times$ state pocket is *on paper* given what this dataset can see. These are structural proxies, not realized outcomes — a row in the Optimistic tier is one where soil, density and window all align with rules of thumb, not one that produced high yield. The next step is to join external outcome data (yield, realized weather at planting) to validate the tiers.

12. [GOING BEYOND] Statistical tests with effect sizes

```
In [20]: # Kolmogorov-Smirnov test for normality on each numeric (Shapiro fails above n=5000).
ks_rows = []
for col in NUM_COLS:
    s = df[col].dropna()
    if len(s) < 30:
        continue
    mu, sd = float(s.mean()), float(s.std(ddof=1))
    if sd == 0:
        continue
    stat, pval = stats.kstest((s - mu) / sd, 'norm')
    ks_rows.append({'variable': col, 'n': len(s), 'KS_D': round(stat, 4), 'p_value': pval, 'skew': round(float(s.skew()), 3), 'kurtosis':
ks_df = pd.DataFrame(ks_rows).set_index('variable')
ks_df
```

Out[20]:

	n	KS_D	p_value	skew	kurtosis
variable					
field_area	319788	0.1987	0.0	14.114	501.622
planted_area_ha	319788	0.4492	0.0	516.529	283317.831
seed_actual_population	319719	0.4949	0.0	424.536	203347.736
seed_actual_population_per_hectare	319719	0.4949	0.0	424.536	203347.736
soil_ph_min	319788	0.3509	0.0	1.161	0.111
soil_ph_max	319788	0.2348	0.0	0.629	-0.725

Reading the table. `KS_D` is the effect size here — the maximum absolute distance between the empirical CDF and the fitted normal. With `n` in the hundreds of thousands, `p-values` will be effectively zero for any non-trivial deviation; the magnitude of `KS_D` (and the skew/kurtosis) is what tells you *how* non-normal the variable is. All six numerics deviate from normality with moderate-to-large `KS_D`, which is why Spearman is the default in §9 and why any simulator layer that assumes normality (e.g. a Gaussian prior on density) must first apply a log or quantile transform.

```
In [21]: # Chi-square test of association between soil_order and crop_name, with Cramer's V as effect size.
pair = df.dropna(subset=['soil_order', 'crop_name']).copy()
pair['soil_order'] = normalize_series(pair['soil_order'])
pair['crop_name'] = normalize_series(pair['crop_name'])
ct = pd.crosstab(pair['soil_order'], pair['crop_name'])
chi2, pval, dof, _ = stats.chi2_contingency(ct)
n = ct.to_numpy().sum()
cramers_v = (chi2 / (n * (min(ct.shape) - 1))) ** 0.5 if min(ct.shape) > 1 else np.nan
pd.Series({'chi2': round(chi2, 2), 'dof': dof, 'p_value': pval, 'n': int(n), 'cramers_v': round(cramers_v, 3)})
```

```
Out[21]: chi2          13776.720
dof              434.000
p_value           0.000
n              319788.000
cramers_v          0.078
dtype: float64
```

Interpretation. The chi-square statistic is expected to be highly significant simply because of the sample size — again, the magnitude to read is **Cramér's V**. Values below 0.1 indicate negligible association; 0.1–0.3 weak; 0.3–0.5 moderate; >0.5 strong. The association here tells us whether crop choice is tightly coupled to soil order in this dataset or whether both vary independently — a useful prior when the simulator wants to generate realistic synthetic rotations.

12b. [GOING BEYOND] Data Quality Scorecard

```
In [22]: # Per-variable scorecard on Completeness / Validity / Consistency / Uniqueness.  
scorecard = dq_scorecard(df)  
scorecard.to_csv(REPORTS / 'dq_scorecard.csv')  
scorecard
```

Out[22]:

	completeness	validity	consistency	uniqueness	dq_score	remediation
variable						
field_uuid	1.0000	1.0000	1.0	0.1386	0.7846	no action; monitor
seed_product_name	0.7601	1.0000	1.0	1.0000	0.9400	add null-flag indicator; do not impute without...
dso_region_seeds	0.8928	1.0000	1.0	1.0000	0.9732	add null-flag indicator; do not impute without...
dso_microregional	0.9704	1.0000	1.0	1.0000	0.9926	no action; monitor
dso_region_cp	0.9704	1.0000	1.0	1.0000	0.9926	no action; monitor
city	0.9710	1.0000	1.0	1.0000	0.9928	no action; monitor
state	0.9710	1.0000	1.0	1.0000	0.9928	no action; monitor
seed_actual_population_per_hectare	0.9998	0.9885	1.0	1.0000	0.9971	no action; monitor
seed_actual_population	0.9998	0.9885	1.0	1.0000	0.9971	no action; monitor
seed_actual_population_uom	0.9986	1.0000	1.0	1.0000	0.9996	no action; monitor
seed_actual_population_per_hectare_raw	0.9998	1.0000	1.0	1.0000	0.9999	no action; monitor
field_area	1.0000	0.9996	1.0	1.0000	0.9999	no action; monitor
seed_actual_population_raw	0.9998	1.0000	1.0	1.0000	0.9999	no action; monitor
planting_date	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
soil_ph_min_raw	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
planted_area_ha_raw	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
field_area_raw	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
planting_machine_name	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
planting_series	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
planting_model	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
planting_maker	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
soil_drainage_class	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
soil_ph_max	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
soil_ph_min	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
soil_texture	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
soil_order	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor

	completeness	validity	consistency	uniqueness	dq_score	remediation
variable						
crop_name	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
concrete_growing_season_name	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
soil_ph_max_raw	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
crop_year	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
growing_season_code	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
climate_account_id	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
field_name	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
seed_product_key	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
planted_area_ha	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor
has_soil_profile	1.0000	1.0000	1.0	1.0000	1.0000	no action; monitor

Interpretation. The bottom of the scorecard (lowest `dq_score`) is where data-stewardship effort pays off most. Typically soil fields and free-text planter fields sit here — the former for completeness, the latter for consistency. Each row carries a named remediation step so the backlog is explicit.

13. [GOING BEYOND] Interactive exploration (Plotly HTML)

```
In [23]: # Interactive 1: rows per state x crop.
pair = df.dropna(subset=['state', 'crop_name']).copy()
pair['crop_name'] = normalize_series(pair['crop_name'])
grouped = pair.groupby(['state', 'crop_name']).size().reset_index(name='rows')
fig1 = px.bar(grouped, x='state', y='rows', color='crop_name', title='Rows per state x crop', barmode='stack')
fig1.write_html(FIGS / 'interactive_state_crop.html', include_plotlyjs='cdn')
fig1
```

Figure saved: reports/figures/interactive_state_crop.html — rows per state × crop (open in browser; no kernel required).

```
In [24]: # Interactive 2: density distribution per crop (seeds/ha only).
mask = df['seed_actual_population_uom'].str.lower().eq('seeds/ha')
plot_df = df.loc[mask, ['crop_name', 'seed_actual_population_per_hectare']].copy()
plot_df['crop_name'] = normalize_series(plot_df['crop_name'])
hi = plot_df['seed_actual_population_per_hectare'].quantile(0.99)
plot_df = plot_df[plot_df['seed_actual_population_per_hectare'] <= hi]
fig2 = px.box(plot_df, x='crop_name', y='seed_actual_population_per_hectare', points=False, title='Density (seeds/ha) per crop – clipped at 99th percentile')
fig2.write_html(FIGS / 'interactive_density_by_crop.html', include_plotlyjs='cdn')
fig2
```

Figure saved: reports/figures/interactive_density_by_crop.html — seed density distribution per crop (open in browser; no kernel required).

```
In [25]: # Interactive 3: scenario tier mix per crop.
tiers_long = (
    scored.groupby(['crop_name_norm', 'tier']).size().reset_index(name='rows')
)
fig3 = px.bar(tiers_long, x='crop_name_norm', y='rows', color='tier', title='Scenario tier composition per crop', barmode='stack', category_order='ascending')
fig3.write_html(FIGS / 'interactive_tier_mix.html', include_plotlyjs='cdn')
fig3
```

The three interactives are saved as standalone HTML under `reports/figures/` :

- `interactive_state_crop.html` — rows per state × crop
- `interactive_density_by_crop.html` — seed density distribution per crop
- `interactive_tier_mix.html` — scenario tier composition per crop

14. Narrative synthesis

Top 5 findings

1. **Density is bimodal because crops are mixed.** The distribution of `seed_actual_population_per_hectare` (\$7, \$13 interactive) has two distinct modes (~70k for corn, ~300k for soy). Any simulator layer that takes density as an input must split by crop first. *Plantability implication:*

planter-calibration recommendations are crop-specific; a cross-crop average is meaningless.

2. **UOM heterogeneity is a silent data-quality trap.** A non-trivial fraction of rows carries a `seed_actual_population_uom` other than `seeds/ha`. Comparing raw `per_hectare` numbers across UOMs mixes apples and oranges (see §6, §11). *Plantability implication:* the simulator must enforce a UOM filter at the density-input stage; otherwise recommended densities will drift.
3. **Soil profile coverage is uneven.** `has_soil_profile` (§4) is True for only a subset of rows; many fields are missing pH or drainage class. *Plantability implication:* the soil-fitness check cannot be applied uniformly. The simulator should return a confidence flag alongside any soil-based recommendation so agronomists know when they are on thin data.
4. **Planting windows are strongly bimodal by season code.** The calendar heatmap (§8) shows tight clustering of SUM activity in Sep–Nov and SAF activity in Jan–Mar. *Plantability implication:* the ± 30 -day window check in the scenario engine (§11) is a defensible filter; it is narrow enough to matter and wide enough to not over-punish legitimate regional variation.
5. **Casing/whitespace drift inflates categorical cardinality.** Free-text fields (drainage class, planter make, soil order) show `drift_ratio < 1` after normalization (§6). *Plantability implication:* any rule that keys off a string literal (e.g. `== 'bem drenado'`) will miss rows. All scenario rules normalize before comparing.

Open questions for Bayer

- **What is the authoritative Embrapa citation** for the pH bands used in the scenario engine? The current reference is generic; a specific DOI would lift `scenario_definitions.md` out of the 'working assumption' box.
- **Why are some rows UOM-free or in `bags/ha`?** Is this a planter-firmware artifact, an account-preference setting, or a deliberate reporting convention?
- **Is `soil_drainage_class` ever revised post-planting?** Our normalization assumes the class is an immutable field attribute.
- **Does Bayer have a canonical `planting_maker` / `planting_model` reference table?** If yes, we can replace normalization with a lookup join and dramatically lower the consistency remediation burden.
- **Can yield / realized outcome be joined in?** Without it, every plantability tier in this notebook is structural, not outcome-validated.

Next-sprint inputs for the simulator

- Canonical reference tables for drainage class, planter make, planter model (build from the normalized distinct values in §6).
- Per-crop density bands (P1 / P50 / P99) computed in §10, plus the UOM whitelist enforced at the ingestion boundary.
- Per-season-code planting window (modal month ± 30 days) from §8.
- `has_soil_profile` indicator propagated as a simulator input so recommendations can flag low-confidence soil slices.
- The four-class anomaly flags from §10 surfaced as row-level exclusion filters at simulator entry.